EV Yük Dengeleme — Sıfırdan Üretime Adım Adım Roadmap

180 Numaralandırılmış Atomik Adım

26 Nisan 2026

İçindekiler

1 Bu Belge Nasıl Kullanılır **2**

2 FAZ 0 — Geliştirme Ortamı Kurulumu (Adım 1–15) **3**

3 FAZ 1 — Mevcut Prototipi Proje Haline Getirme (Adım 16–30) **8**

4 FAZ 2 — Maliyet & Tarife Modülü (Adım 31–40) **13**

5 FAZ 3 — OCPP'yi Sıfırdan Öğrenme (Adım 41–60) **16**

6 FAZ 4 — Donanım Lab Kurulumu (Adım 61–75) **21**

7 FAZ 5 — Backend MVP (Adım 76–100) **25**

8 FAZ 6 — Telemetri Pipeline (Adım 101–115) **31**

9 FAZ 7 — Yük Tahmini (Adım 116–140) **35**

10 FAZ 8 — Algoritmayı Production'a Port (Adım 141–150) **41**

11 FAZ 9 — Operatör Dashboard (Adım 151–165) **44**

12 FAZ 10 — Bulut Deploy + CI/CD (Adım 166–175) **47**

13 FAZ 11 — Pilot Saha Kurulumu (Adım 176–180) **50**

14 Sonraki 5 Adım — Faz 11 sonrası (özet) **52**

15 Genel Notlar **52**

15.1 Paralelize Edilebilen Adımlar (3 Kurucu) . . . 52

15.2 Hangi Adımı Ne Zaman Yapacaksın — Bağımlılık Grafiği . . . 52

15.3 Bittiği Nasıl Anlaşılır (Genel) . . . 52

15.4 Sıkıştığında Ne Yaparsın . . . 52

15.5 Final Hatırlatma . . . 53

1 Bu Belge Nasıl Kullanılır

Bu roadmap `simulation_v3.py` prototipinden ticari ürüne giden yolu **180 sıralı adıma** bölmüştür. Her adım atomik (1-3 saatlik somut iş), bağımlılığı belli, başarı kriteri tanımlı.

Her adım şu şablona göredir:

- **Niye:** bu adım niye var.
- **Önce:** hangi adımları bitirmen gerek.
- **Ne yapacaksın:** somut aksiyonlar.
- **Bittiği nasıl anlaşılır:** kabul kriteri.

Stratejik kullanım:

- **Sırayı değiştirme**, bağımlılıklar gerçek.
- Her adım sonunda Git commit ("adım N: özet").
- 3 kurucu ile bazı adımlar paralel yürür — her fazın başında "paralelize edilebilir" adımlar listelenmiştir.
- Tıkanırsan: bir önceki adıma geri dön, kabul kriterini yeniden kontrol et.

Faz dağılımı:

- **Faz 0** (Adım 1–15): Geliştirme ortamı sıfırdan kurulumu.
- **Faz 1** (Adım 16–30): Mevcut prototipi "proje" haline getirme.
- **Faz 2** (Adım 31–40): Maliyet/tarife modülü.
- **Faz 3** (Adım 41–60): OCPP'yi sıfırdan öğrenme + lab.
- **Faz 4** (Adım 61–75): Donanım kurulumu (Vestel + Janitza + Gateway).
- **Faz 5** (Adım 76–100): Backend MVP (FastAPI + DB + API).
- **Faz 6** (Adım 101–115): Telemetri pipeline + akıllı sayaç.
- **Faz 7** (Adım 116–140): Yük tahmini (veri → ML model → üretim).
- **Faz 8** (Adım 141–150): Algoritmayı production'a port.
- **Faz 9** (Adım 151–165): Operatör dashboard (Next.js).
- **Faz 10** (Adım 166–175): Bulut deploy + CI/CD.
- **Faz 11** (Adım 176–180): Pilot sahaya kurulum.

2 FAZ 0 — Geliştirme Ortamı Kurulumu (Adım 1–15)

*Bu fazda hiçbir kod yazmıyorsun — aletleri kurar, hesapları açar, ekibi senkronlarsın. Atlamak **çok pahalıya patlıyor**; her adımı tamamla.*

Adım 1. Python 3.12 kurulumu (her kurucu)

Niye: Tüm kod buradan çalışacak; sürüm uyumsuzluğu en sık ekip kavgası sebebidir.
Önce: —
Ne yapacaksın:

- Windows: python.org → 3.12.x installer indir → "Add Python to PATH" işaretli kur.
- Mac: `brew install python@3.12`
- Linux: `sudo apt install python3.12 python3.12-venv`

Bittiği nasıl anlaşılır: Terminalde `python -version` → `Python 3.12.x`.

Adım 2. Git kurulumu + global config (her kurucu)

Niye: Versiyon kontrolü olmadan ekip çalışması imkansız.
Önce: —
Ne yapacaksın:

- Windows: git-scm.com → installer.
- `git config -global user.name "Adın Soyadın"`
- `git config -global user.email "sen@example.com"`
- `git config -global init.defaultBranch main`
- SSH key oluştur: `ssh-keygen -t ed25519 -C "sen@example.com"`, çıktı `~/.ssh/id_ed25519.pub`'u GitHub'a ekle.

Bittiği nasıl anlaşılır: `ssh -T git@github.com` → "Hi <kullanıcı>! You've successfully authenticated."

Adım 3. VS Code + temel eklentiler (her kurucu)

Niye: Tek IDE ekipte standart; format ve linter farklı olunca commit savaşı çıkar.
Önce: —
Ne yapacaksın:

- code.visualstudio.com → kur.
- Eklentiler: Python (Microsoft), Pylance, Black Formatter, Ruff, GitLens, Docker, Remote SSH, Even Better TOML.
- Settings: `"editor.formatOnSave": true`, `"python.formatting.provider": "black"`.

Bittiği nasıl anlaşılır: Boş `.py` dosyasında `print("hi")` yaz, kaydet, otomatik formatlanması.

Adım 4. GitHub organizasyonu & private repo aç

Niye: Kod org altında olunca founder ayrılığı durumunda repo kişide kalmaz.
Önce: Adım 2.
Ne yapacaksın:

- GitHub → New organization → Free plan başla.

- Org adı: şirket adınla aynı (henüz yoksa kod adı koy, sonra yeniden adlandır).
- Repo oluştur: `<org>/evlb-core` (private). README + .gitignore (Python) + MIT (sonra değiştirebilirsin) seç.
- Üç kurucuyu admin olarak ekle.

Bittiği nasıl anlaşılır: 3 kurucu da repo URL'sine erişebiliyor.

Adım 5. Repo klonla & mevcut prototipi taşı

Niye: `F:/LoadBalancing/simulation_v3.py` versiyon kontrolüne girmeli.
Önce: Adım 4.
Ne yapacaksın:

- `git clone git@github.com:<org>/evlb-core.git`
- Mevcut dosyaları (`simulation_v3.py`, `dataset.json`, son tex/png çıktıları, README) yeni repo dizinine kopyala.
- `git add . && git commit -m "feat: import existing simulation prototype" && git push`

Bittiği nasıl anlaşılır: GitHub web arayüzünde `simulation_v3.py` görünüyor.

Adım 6. Branch protection & PR kuralları

Niye: Hiçbir kurucu doğrudan `main`'e push edemesin; her değişiklik PR'dan geçsin.
Önce: Adım 5.
Ne yapacaksın:

- Repo Settings → Branches → `main` için "Require pull request before merging" aç.
- "Require approvals" = 1.
- "Require status checks" (CI eklendiğinde geri gel).
- `develop` branch oluştur, `main` = release branch olsun.

Bittiği nasıl anlaşılır: `git push origin main` reddediliyor; PR açmak zorundasın.

Adım 7. Python virtual environment (lokal)

Niye: Sistem Python'una paket yüklemek başka projelerini bozar.
Önce: Adım 1, 5.
Ne yapacaksın:

- Repo dizininde: `python -m venv .venv`
- Windows: `.venv\Scripts\activate`
- Mac/Linux: `source .venv/bin/activate`
- `pip install -upgrade pip wheel`

Bittiği nasıl anlaşılır: Komut satırında `(.venv)` prefix görünüyor; `which python` → repo içinde.

Adım 8. Mevcut bağımlılıkları tespit et + requirements.txt

Niye: Şu an hangi paketler kullanılıyor netleşmeli; sonra çoğalacak.
Önce: Adım 7.
Ne yapacaksın:

- `pip install numpy pandas matplotlib openpyxl`
- `pip freeze > requirements.txt`
- `python simulation_v3.py -scenario avm_medium` → çalışıyor mu kontrol.

Bittiği nasıl anlaşılır: `simulation_v3.py` hatasız çalışıyor, Excel + PNG çıkışları üretiliyor.

Adım 9. .gitignore tamamla

Niye: `.venv`, derleme çıktıları, `__pycache__` repo'yu kirletmemeli.
Önce: Adım 5.
Ne yapacaksın:

- GitHub Python.gitignore kopyala.
- Ek satırlar: `*.png`, `*.xlsx`, `*.pdf`, `.aux`, `.log`, `.toc`, `.out`, `output/`, `*.bak`.
- Ek: `.idea/`, `.vscode/` (IDE).

Bittiği nasıl anlaşılır: `git status` → `.venv` ya da `__pycache__` listede yok.

Adım 10. Black + Ruff + isort konfigürasyonu (`pyproject.toml`)

Niye: Otomatik format= code review'da kavga yok.
Önce: Adım 7.
Ne yapacaksın:

- Repo köküne `pyproject.toml`:

```
[tool.black]
line-length = 100
target-version = ["py312"]

[tool.ruff]
line-length = 100
target-version = "py312"
select = ["E", "F", "I", "N", "B", "UP"]

[tool.isort]
profile = "black"
```

- `pip install black ruff isort`
- `black simulation_v3.py && ruff check simulation_v3.py`

Bittiği nasıl anlaşılır: `black -check .` → hata yok.

Adım 11. Pre-commit hooks

Niye: Commit anında otomatik format/lint çalışsın; "yarın bakarım" tuzağı engellensin.
Önce: Adım 10.
Ne yapacaksın:

- `pip install pre-commit`
- Repo köküne `.pre-commit-config.yaml`:

```
repos:
  - repo: https://github.com/psf/black
    rev: 24.10.0
    hooks: [{id: black}]
  - repo: https://github.com/charliermarsh/ruff-pre-commit
```

```
    rev: v0.7.0
    hooks: [{id: ruff, args: [--fix]}]
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
```

- `pre-commit install`

Bittiği nasıl anlaşılır: `git commit` sırasında hooklar çalışıyor.

Adım 12. GitHub Actions CI iskeleti

Niye: PR'da otomatik test çalışmadan kabul etme.
Önce: Adım 11.
Ne yapacaksın:

- `.github/workflows/ci.yml` oluştur:

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r requirements.txt black ruff pytest
      - run: black --check .
      - run: ruff check .
      - run: pytest -v
```

- Test dosyası daha yok; pytest fail eder, sonraki adımda eklenecek.

Bittiği nasıl anlaşılır: GitHub Actions sekmesinde workflow görünüyor.

Adım 13. İlk pytest testi

Niye: CI yeşil olsun; alışkanlık başlasın.
Önce: Adım 12.
Ne yapacaksın:

- `pip install pytest && pip freeze > requirements.txt`
- `tests/test_smoke.py`:

```
def test_smoke():
    assert 1 + 1 == 2
```

- Commit + push.

Bittiği nasıl anlaşılır: GitHub Actions → CI yeşil.

Adım 14. Founders' Agreement + ESOP

Niye: Kod yazmadan önce hukuki temeli at; ileride ortak ayrılığında repo kavgası çıkar.
Önce: —
Ne yapacaksın:

- Hukuki danışmanla 1 toplantı (Esin Avukatlık, Pekin & Bayar, Akol Hukuk).
- Konular: founder pay oranı, vesting (4 yıl, 1 yıl cliff), ESOP havuzu (%10–15), yapım hakkı (founder ayrılırsa repo'yu çekemez), tahkim maddesi.
- İmzalı PDF Drive/Notion'a; hard copy notere onay.

Bittiği nasıl anlaşılır: 3 kurucu ile imzalı + noter tasdikli sözleşme dosyada.

Adım 15. Notion / Linear (proje yönetimi)

Niye: 180 adımı bireysel takip etmek imkansız; ekipte tek araç olmalı.
Önce: —
Ne yapacaksın:

- Linear (issue tracker) free plan, ya da Notion (notes + roadmap).
- Bu doküman → Notion sayfasına; her adım = bir Linear issue.
- Tag: `P0/P1/P2`, owner: 1 kurucu.

Bittiği nasıl anlaşılır: 180 adım Linear'da issue olarak görünüyor.

3 FAZ 1 — Mevcut Prototipi Proje Haline Getirme (Adım 16–30)

Bu fazda yeni özellik yazmıyorsun, mevcut tek dosyayı yeniden organize ediyorsun. Sıkıcı ama teknik borç vermeyen tek yol bu.

Adım 16. Hedef paket yapısı tasarımı

Niye: 880 satırlık tek dosya idare etmiyor; modüllere böleceğiz.
Önce: Adım 7–13.
Ne yapacaksın:

- Hedef yapı:

```
src/evlb/
  __init__.py
  models.py # EV, EVModel, ChargingStation
  scenarios.py # ScenarioConfig, Scenarios, EnvironmentProfile
  controllers/
    __init__.py
    base.py # BaseController (ortak step)
    unmanaged.py
    managed.py
    srpt.py
    water_filling.py
    dynamic_fair.py
  simulation.py # Simulation runner
  data/
    arrival_generator.py
    background_load.py
  io/
    excel_export.py
    dashboard.py
    dataset_io.py
  config.py # config dataclasses
  cli.py # argparse  main()
```

Bittiği nasıl anlaşılır: Diyagram dokümante edilmiş, ekip onaylı.

Adım 17. `src/` layout + pyproject.toml package definition

Niye: Modern Python paket olarak yüklenebilmeli (`pip install -e .`).
Önce: Adım 16.
Ne yapacaksın:

- `pyproject.toml`'a ekle:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "evlb"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "numpy>=1.26",
    "pandas>=2.2",
    "matplotlib>=3.8",
    "openpyxl>=3.1",
]

[tool.setuptools.packages.find]
where = ["src"]
```

- `pip install -e .[dev]` (development install).

Bittiği nasıl anlaşılır: `python -c "import evlb; print(evlb.__file__)"` → src/evlb/__init__.py path.

Adım 18. `models.py` oluştur

Niye: Veri sınıflarını ayrı dosya= test edilebilir, importu hızlı.
Önce: Adım 17.
Ne yapacaksın:

- `simulation_v3.py` satır 29–133 (Enum'lar, EVModel, EV, ChargingStation, dataclass'lar) → `src/evlb/models.`
- Kayıp olanları (StationType, EVState) da taşı.

Bittiği nasıl anlaşılır: `python -c "from evlb.models import EV, EVModel"` → hata yok.

Adım 19. `config.py` (Scenario/Environment/Grid configleri)

Niye: Konfigürasyon ile veri modeli ayrı endişeler.
Önce: Adım 18.
Ne yapacaksın:

- Satır 139–235 (`EnvironmentProfile, GridConfig, ArrivalPattern, FleetProfile, StationLayout, ScenarioConfig`) → `src/evlb/config.py`.
- `from_dict / to_dict` metotlarını koru.

Bittiği nasıl anlaşılır: JSON serileştirme test edildi.

Adım 20. `data/` alt-paketi (üreticiler)

Niye: Sentetik veri üretimi tek yerde; ileride gerçek veri loader buraya gelecek.
Önce: Adım 19.
Ne yapacaksın:

- Satır 251–283: `ArrivalGenerator` → `src/evlb/data/arrival_generator.py`.
- Satır 285–300: `BackgroundLoadGenerator` → `src/evlb/data/background_load.py`.
- Satır 241–249: `_DEFAULT_EV_MODELS` → `src/evlb/data/ev_models.py`.

Bittiği nasıl anlaşılır: İmportlar çalışıyor; `ArrivalGenerator(...).generate_arrivals(rng)` kullanılabiliyor.

Adım 21. `scenarios.py`

Niye: 5 senaryo (avm/ofis/otel/hastane/havalimanı) tek yerde tanımlı.
Önce: Adım 20.
Ne yapacaksın:

- Satır 302–463: `Scenarios` sınıfı → `src/evlb/scenarios.py`.

Bittiği nasıl anlaşılır: `Scenarios.avm_medium()` sonuç veriyor.

Adım 22. `BaseController` soyut sınıfı

Niye: 5 kontrolcüde `step()` aynı; sadece `allocate_power()` değişiyor. Refactor zamanı.
Önce: Adım 17.
Ne yapacaksın:

- `src/evlb/controllers/base.py`:

```
from abc import ABC, abstractmethod

class BaseController(ABC):
    def __init__(self, stations, limit_policy, bg_load):
        self.stations = stations
        self.policy = limit_policy
        self.bg_load = bg_load
        self.queue = []
        self.power_log = []
        self.limit_log = []
        self.completed = []
        self.timeline_log = []

    @abstractmethod
    def allocate_power(self, minute: int) -> dict[str, float]:
        ...

    def step(self, minute: int):
        # mevcut step() icindekileri buraya tasi
        ...
```

Bittiği nasıl anlaşılır: `BaseController`'dan abstract method olmadan instance oluşturulamıyor (`TypeError`).

Adım 23. 5 kontrolcüyü ayrı dosyalara böl + BaseController extend

Niye: Tek tek dosya= test ve gözden geçirme kolay.
Önce: Adım 22.
Ne yapacaksın:

- `controllers/unmanaged.py, managed.py, srpt.py, water_filling.py, dynamic_fair.py`.
- Her biri `from .base import BaseController` ile genişlet.
- `controllers/__init__.py` → tüm sınıfları import.

Bittiği nasıl anlaşılır: `from evlb.controllers import SRPTController` çalışıyor.

Adım 24. `simulation.py` ayrı modül

Niye: Runner ayrı yerde; entegrasyon testleri buradan başlar.
Önce: Adım 23.
Ne yapacaksın:

- Satır 588–603: `Simulation` class → `src/evlb/simulation.py`.

Bittiği nasıl anlaşılır: `Simulation(ctrl, sched).run()` çalışıyor.

Adım 25. `io/` alt-paketi (export)

Niye: Dashboard & Excel; çıktı katmanı izole.
Önce: Adım 24.
Ne yapacaksın:

- `io/excel_export.py`: `build_station_matrix, export_comparative_excel, export_multi_controller_ex`
- `io/dashboard.py`: `ExecutiveDashboard`.
- `io/dataset_io.py`: dataset.json okuma/yazma helper'ları.

Bittiği nasıl anlaşılır: İmport zinciri temiz, dairesel bağımlılık yok.

Adım 26. `cli.py` & entry point

Niye: `python -m evlb` ile çalışsın; eski main `simulation_v3.py` yerine geçsin.
Önce: Adım 25.
Ne yapacaksın:

- `src/evlb/cli.py`'a eski `main()` fonksiyonunu taşı.
- `pyproject.toml`'a:

```
[project.scripts]
evlb = "evlb.cli:main"
```

- `pip install -e .` sonrası `evlb -scenario avm_medium` → çalışmalı.

Bittiği nasıl anlaşılır: CLI komutu eski script'le aynı çıktıyı veriyor.

Adım 27. Birim test (her controller için)

Niye: Refactor sonrası eskiyle aynı sonucu üretmek garanti olmalı.
Önce: Adım 26.
Ne yapacaksın:

- `tests/controllers/test_dynamic_fair.py`: belirli input → beklenen `allocs` çıktı.
- `tests/test_simulation_regression.py`: `dataset.json` ile çalıştır, eski Excel çıktısının aynısını üretiyor mu (golden file).
- `pytest -v` → tüm yeşil.

Bittiği nasıl anlaşılır: `pytest -cov=evlb` → %70+ coverage; CI yeşil.

Adım 28. Eski `simulation_v3.py`'yi sil + commit

Niye: Tek doğru kod kaynağı; çift bakım= bug.
Önce: Adım 27.
Ne yapacaksın:

- Eski Excel/PNG çıktıları `output/` dizinine taşı, .gitignore'a ekle.
- `git rm simulation_v3.py`
- `git commit -m "refactor: remove monolithic simulation_v3, use evlb package"`

Bittiği nasıl anlaşılır: Repo'da `simulation_v3.py` yok; `evlb` paketi tüm işi yapıyor.

Adım 29. README'yi güncelle

Niye: 6 ay sonra senin de hatırlamayacağın komutlar; yeni hire 5 dakikada anlasın.
Önce: Adım 28.
Ne yapacaksın:

- Bölümler: Proje nedir, kurulum (`pip install -e .`), kullanım (`evlb -scenario X`), test (`pytest`), proje yapısı, lisans.

Bittiği nasıl anlaşılır: README ile yeni biri 10 dakikada simülasyonu çalıştırabiliyor.

Adım 30. İlk `v0.1.0` tag'i

Niye: Refactor öncesi prototipin çalışan haline geri dönebilmek için.
Önce: Adım 29.
Ne yapacaksın:

- `git tag -a v0.1.0 -m "Refactored prototype baseline"`
- `git push -tags`
- GitHub Releases sayfasında release notes (kabaca yapılanlar listesi).

Bittiği nasıl anlaşılır: GitHub Releases → v0.1.0 görünüyor.

4 FAZ 2 — Maliyet & Tarife Modülü (Adım 31–40)

Adım 31. TEDAŞ tarifesini araştır

Niye: Müşteriye "%X tasarruf" demek için TL bilmen gerek.
Önce: —
Ne yapacaksın:

- EPDK tarife duyurularını oku: epdk.gov.tr/elektriktarifeler.
- Sanayi/ticari kademeli tarife (Gündüz 06–17, Puant 17–22, Gece 22–06).
- Reaktif limit (%20 endüktif, %15 kapasitif), aşımda ceza.
- Güç bedeli (TL/kW.ay) — OG müşteri için.
- Dağıtım + iletim bedeli, kaybı dahil.
- KDV %20.

Bittiği nasıl anlaşılır: 1 sayfalık özet (Notion).

Adım 32. `tariff/` modülü iskeleti

Niye: Kod tabanlı tarife= sözleşmede ROI cümlesi.
Önce: Adım 31.
Ne yapacaksın:

- `src/evlb/tariff/__init__.py`
- `tariff/models.py`: `TariffSchedule` dataclass (TOU dilimleri, fiyatlar).
- `tariff/tedas_2026.json`: güncel fiyatlar.

Bittiği nasıl anlaşılır: `TariffSchedule.from_json("tedas_2026.json")` çalışıyor.

Adım 33. Aktif enerji maliyet hesaplayıcısı

Niye: kWh × TL/kWh= ana maliyet kalemi.
Önce: Adım 32.
Ne yapacaksın:

- `tariff/calculator.py`:

```
def energy_cost(power_kw_per_min, tariff: TariffSchedule):
    """power_kw_per_min: 1440 elemanli np.ndarray"""
    energy_kwh = power_kw_per_min / 60.0 # dakika -> saat
    cost = 0.0
    for minute, kwh in enumerate(energy_kwh):
        rate = tariff.rate_at_minute(minute)
        cost += kwh * rate
    return cost
```

- Test: 24 sa sabit 100 kW → toplam 2400 kWh → tarifeli toplam.

Bittiği nasıl anlaşılır: Test geçiyor; manuel kontrol matematiği uyuyor.

Adım 34. Demand charge hesabı

Niye: OG müşteride "aylık 15 dk maks güç" büyük kalem; bizim algoritma bunu düşürüyor.
Önce: Adım 33.
Ne yapacaksın:

- 15 dk pencere maks güç hesabı (rolling).
- `demand_charge_cost(power_log, tl_per_kw_month)` → en yüksek 15-dk avg × tarife.

Bittiği nasıl anlaşılır: Yapay test girdisinde elde-hesap aynı.

Adım 35. Reaktif enerji limit kontrolü

Niye: Şarj cihazları reaktif çekebilir; limit aşımı= ceza.
Önce: Adım 33.
Ne yapacaksın:

- Şu an reaktif veri yok (sentetik); skeleton fonksiyon yaz: `reactive_penalty(active_kwh, reactive_kvarh, limit=0.20)`.
- Faz 6'da gerçek Janitza verisi eklenecek; o zaman doldurulur.

Bittiği nasıl anlaşılır: Skeleton var, TODO commenti.

Adım 36. ROI Calculator

Niye: Müşteri "geri ödeme süresi kaç ay?" diye soruyor; cevap hazır olmalı.
Önce: Adım 33–35.
Ne yapacaksın:

- `tariff/roi.py`:

```
def simulate_roi(baseline_log, optimized_log, tariff,
                capex_tl, monthly_saas_tl):
    monthly_saving = (
        energy_cost(baseline_log, tariff)
        + demand_charge_cost(baseline_log, ...)
        - energy_cost(optimized_log, tariff)
        - demand_charge_cost(optimized_log, ...)
    ) * 30 # gunluk -> aylik
    payback_months = capex_tl / max(0.001, monthly_saving - monthly_saas_tl)
    return {"monthly_saving": monthly_saving, "payback": payback_months}
```

Bittiği nasıl anlaşılır: 5 senaryo için `evlb compare-roi` → tablo basıyor.

Adım 37. ROI raporunu Excel'e bağla

Niye: Müşteri toplantısında dashboard + Excel sayfasında somut TL.
Önce: Adım 36.
Ne yapacaksın:

- `export_multi_controller_excel` fonksiyonuna ROI sütunu ekle.
- Aylık tasarruf TL, geri ödeme süresi.

Bittiği nasıl anlaşılır: Excel'de yeni ROI sütunu.

Adım 38. ROI dashboard panel

Niye: PNG'da görsel etkili.
Önce: Adım 36.
Ne yapacaksın:

- `ExecutiveDashboard`'a yeni panel: bar chart "Aylık Tasarruf (TL)" + payback period yazısı.

Bittiği nasıl anlaşılır: PNG dashboard'da panel görünüyor.

Adım 39. Tarife versiyonlama

Niye: EPDK her çeyrek günceller; eski sözleşmelerde eski tarife kullanılır.
Önce: Adım 32.
Ne yapacaksın:

- `tariff/tedas_2026_q1.json, tedas_2026_q2.json`.
- `TariffSchedule.load_for_date(d: date)` → uygun versiyon.

Bittiği nasıl anlaşılır: Hangi tarifeyle hesaplandığı log'a yazılıyor.

Adım 40. İhracat: "Müşteri ROI Raporu" PDF

Niye: Customer discovery toplantısında bırakılacak somut materyal.
Önce: Adım 36–39.
Ne yapacaksın:

- LaTeX template (basit) → `evlb roi-report -site X` → `musteri_X_roi.pdf`.
- İçerik: senaryo özeti, baseline TL, optimized TL, tasarruf, geri ödeme.

Bittiği nasıl anlaşılır: 1 saatlik AVM senaryosu için PDF rapor üretiliyor.

5 FAZ 3 — OCPP'yi Sıfırdan Öğrenme (Adım 41–60)

Bilgin sıfır demiştin — bu fazda "Open Charge Point Protocol"ü tanıyacaksın. Spec oku, lokal çalıştır, mesaj akışını gör.

Adım 41. OCPP nedir? — 1 sayfa özet

Niye: Her şey burada başlıyor; net anlamadan kod yazma.
Önce: —
Ne yapacaksın:

- Open Charge Alliance — OCPP 1.6 sayfasını oku.
- Kavramlar: Charge Point (CP), Central System (CSMS), heartbeat, transaction, MeterValues, ChargingProfile.
- Notion'da 1 sayfalık özet yaz (kendin için).

Bittiği nasıl anlaşılır: "`StartTransaction` ne işe yarar?" sorusuna 30 saniyede cevap verebiliyorsun.

Adım 42. OCPP 1.6 spec PDF'ini indir + flick et

Niye: Mesaj formatlarını referans olarak kullanacaksın.
Önce: Adım 41.
Ne yapacaksın:

- OCA üyesi değilsen halka açık: mobilityhouse/ocpp docs.
- Ya da OCA üyelik ($1500/yıl) → resmi spec PDF.
- İçindekilerle dolaş; her mesajın işlevini bir cümlede özetle.

Bittiği nasıl anlaşılır: 30 mesajın 10'unun ne işe yaradığını biliyorsun.

Adım 43. WebSocket nedir? — network bilgisi

Niye: OCPP WebSocket üstünde çalışır; HTTP'den farkını bilmen gerek.
Önce: —
Ne yapacaksın:

- MDN WebSocket guide: 30 dakika oku.
- Anahtar: persistent connection, full-duplex, message-based.
- Lokalde basit deneme: `websocat` CLI tool ile bağlan.

Bittiği nasıl anlaşılır: HTTP vs WebSocket farkını anlatabiliyorsun.

Adım 44. JSON-RPC ve OCPP-J formatı

Niye: OCPP mesajları JSON; `[2, "msgId", "Action", {payload}]` formatı.
Önce: Adım 42.
Ne yapacaksın:

- Spec'in 3.4 bölümü (Message Format).
- 4 tip: CALL (2), CALLRESULT (3), CALLERROR (4), CALLPING (heartbeat değil).
- Örnek BootNotification CALL/CALLRESULT manuel yazıp anla.

Bittiği nasıl anlaşılır: BootNotification mesajını elinle JSON yazabiliyorsun.

Adım 45. `mobilityhouse/ocpp` kütüphanesini kur

Niye: Açık kaynak Python OCPP server/client; tekerlek icat etme.
Önce: Adım 7, 44.
Ne yapacaksın:

- `pip install ocpp websockets`
- GitHub repo'yu klonla: `git clone https://github.com/mobilityhouse/ocpp.git`
- `examples/v16/charge_point.py` ve `examples/v16/central_system.py` oku.

Bittiği nasıl anlaşılır: İki dosyada da hangi handler nerede biliyorsun.

Adım 46. Lokalde örnek CSMS çalıştır

Niye: Sahte bir charger bağlanmadan önce server hazır olmalı.
Önce: Adım 45.
Ne yapacaksın:

- `python examples/v16/central_system.py`
- Çıktı: "Server starting on `ws://0.0.0.0:9000`".

Bittiği nasıl anlaşılır: Server log'unda hata yok.

Adım 47. Sahte charger çalıştır + ilk `BootNotification`

Niye: Mesaj akışını canlı görmek = anlamak.
Önce: Adım 46.
Ne yapacaksın:

- Yeni terminal: `python examples/v16/charge_point.py`
- CSMS log'unda: `» [2,...,"BootNotification",{...}]`
  `« [3,...,{"status":"Accepted"}]`.

Bittiği nasıl anlaşılır: İlk gerçek mesaj akışını gördün.

Adım 48. `Heartbeat` ve `StatusNotification`

Niye: Charger "hayatta" demeyi ve durum değişimini bildirmeyi öğreniyor.
Önce: Adım 47.
Ne yapacaksın:

- Sahte charger'a heartbeat döngüsü ekle (her 30 sn).
- Manuel: socket çıkarılmış simülasyonu için `StatusNotification(status="Available")`.

Bittiği nasıl anlaşılır: Server log'unda her 30 sn heartbeat görünüyor.

Adım 49. Authorize / StartTransaction / StopTransaction akışı

Niye: Şarj seansının başı/sonu; bu olmadan paralı şarj olmaz.
Önce: Adım 48.
Ne yapacaksın:

- Sahte charger:

  - RFID = "ABC123" ile `Authorize`.
  - CSMS `Accepted` dönüyor.
  - `StartTransaction(meterStart=12345)`.
  - `StopTransaction(meterStop=12500)`.
- CSMS log incele.

Bittiği nasıl anlaşılır: 4 mesaj sırayla görünüyor; CSMS transaction_id atıyor.

Adım 50. `MeterValues` mesajı

Niye: Anlık güç telemetrisi; algoritmanın asıl input'u.
Önce: Adım 49.
Ne yapacaksın:

- Sahte charger'da her 60 sn `MeterValues` gönder. Sampled values: `Power.Active.Import, Energy.Active.Impor` `Voltage, Current.Import`.
- `measurand`, `unit`, `location` alanlarını anla.

Bittiği nasıl anlaşılır: Server'da telemetry değerleri görünüyor (sahte de olsa).

Adım 51. `SetChargingProfile` — bizim için kritik mesaj

Niye: Ana mesleğimiz bu — gücü düşürme/yükseltme komutu.
Önce: Adım 50.
Ne yapacaksın:

- CSMS'den charger'a `SetChargingProfile` gönder:
  - `chargingProfilePurpose`: `TxProfile` (transaction-bazlı).
  - `chargingProfileKind`: `Absolute`.
  - `chargingSchedule`: `startSchedule, chargingRateUnit="W"`, `chargingSchedulePeriod=[{startPeriod:0` `limit:11000}]`.
- Sahte charger'ın bu mesajı aldığını log'la.

Bittiği nasıl anlaşılır: 11 kW limit gönderildi, sahte charger `Accepted` döndü.

Adım 52. ChargingProfile yapısını derinleşmeli oku

Niye: Statik limit basit; dinamik (zaman-değişken) profil ileride gerekecek.
Önce: Adım 51.
Ne yapacaksın:

- Spec'ın `Smart Charging` eki (Annex B in 1.6).
- Stack levels (1-9), `TxDefaultProfile` vs `TxProfile` farkı.
- `schedulePeriod` array ile zaman-değişken: 15 dk 11 kW, sonra 15 dk 22 kW.

Bittiği nasıl anlaşılır: Multi-period schedule yazıp gönderebiliyorsun.

Adım 53. `RemoteStartTransaction` & `RemoteStopTransaction`

Niye: Kullanıcı app'ten "başlat" demeden charger'ı uzaktan başlatma.
Önce: Adım 49.
Ne yapacaksın:

- CSMS'den `RemoteStartTransaction(idTag, connectorId)`.
- Sahte charger `Accepted` → kendiliğinden `StartTransaction`.

Bittiği nasıl anlaşılır: Mesaj zinciri akıyor.

Adım 54. Birden çok charger aynı server'a

Niye: Multi-charger yönetimi gerçek senaryomuz.
Önce: Adım 50.
Ne yapacaksın:

- 5 sahte charger lansé et farklı portlardan.
- CSMS'de her birinin kayıtlı olduğunu gör.
- Her birine farklı `SetChargingProfile` gönder.

Bittiği nasıl anlaşılır: 5 charger 5 farklı limit alıyor.

Adım 55. OCPP 2.0.1 farklarını oku (yüzeysel)

Niye: İleride 2.0 desteği gerekli; ne değişti bilmek iyi.
Önce: Adım 41.
Ne yapacaksın:

- Mobilityhouse 2.0.1 docs.
- Anahtar farklar: `TransactionEvent` tek mesaj (Start/Stop birleşti), security profile zorunlu, ISO 15118 desteği, modüler özellik (functional groups).
- Tablo: 1.6 mesaj → 2.0.1 karşılığı.

Bittiği nasıl anlaşılır: Tablo Notion'da, ekiple paylaşıldı.

Adım 56. `SteVe` (Java) çalıştır — referans CSMS

Niye: Açık kaynak hazır CSMS; admin paneli olan; öğrenmek için iyi referans.
Önce: —
Ne yapacaksın:

- `git clone https://github.com/steve-community/steve.git`
- Docker ile çalıştır: `docker compose up`.
- Web UI: `localhost:8180/manager`.
- Sahte charger oluştur, mesaj akışını UI'dan izle.

Bittiği nasıl anlaşılır: SteVe UI'da heartbeat'leri görüyorsun.

Adım 57. OCPP test tool'u (OCA test tool ya da MaEVe)

Niye: Üreticinin charger'ı OCPP'ye uyuyor mu test etmek için.
Önce: Adım 47.
Ne yapacaksın:

- ThoughtWorks MaEVe klonla.
- Test senaryolarını oku.
- Bizim yazılımımızla bunu çalıştır → uyumluluk haritası çıkar.

Bittiği nasıl anlaşılır: Test çıktısı dosyaya kaydedildi.

Adım 58. Kendi minimal OCPP server'ımızı yaz

Niye: Sonunda `evlb`'ye entegre etmemiz lazım; `mobilityhouse/ocpp`'yi kullanarak kendi sarmalayıcımızı yaz.
Önce: Adım 47–50.
Ne yapacaksın:

- `src/evlb/ocpp/` alt-paketi.
- `ocpp/server.py`: WebSocket endpoint, charger kayıt yönetimi.
- `ocpp/charger.py`: charger sınıfı, `send_charging_profile(limit_kw)` metodu.
- Sahte charger'ları artık `evlb`'ye bağla.

Bittiği nasıl anlaşılır: `python -m evlb.ocpp.server` → 5 sahte charger bağlanıyor.

Adım 59. OCPP server'ı algoritma çıkışıyla bağla

Niye: Sim'deki `allocate_power` sonucunu gerçek `SetChargingProfile` mesajına dönüştürmek.
Önce: Adım 59 ve Faz 1.
Ne yapacaksın:

- Bir "loop" yaz: her 60 sn → `DynamicFairController.allocate_power` → her charger'a `SetChargingProfile`.
- `MeterValues`'i sahte charger'dan oku, EV'nin `current_soc`'unu güncelle.

Bittiği nasıl anlaşılır: 30 dk simülasyon: charger'a değişen limitler gidiyor; SoC artıyor.

6 FAZ 4 — Donanım Lab Kurulumu (Adım 61–75)

Sahte charger'ın ötesine geç — gerçek charger'a bağlan.

Adım 60. Lab fiziksel ortam

Niye: Charger 380V trifaze elektrik ister; ev prizine bağlayamazsın.
Önce: —
Ne yapacaksın:

- Co-working / küçük ofis kirala (50K–150K TL/ay alanına göre).
- Trifaze 32A hat çek (elektrikçi: 5–15K TL).
- AG pano + sigorta + topraklama.

Bittiği nasıl anlaşılır: Elektrikçi raporu (TEDAŞ uyumu).

Adım 61. Charger satın al / kirala

Niye: Gerçek üzerinde test olmadan müşteriye gidemezsin.
Önce: Adım 61.
Ne yapacaksın:

- Vestel ile temas: "demo birimi ödünç + satış kanalı potansiyeli."
- Alternatif: 2. el ABB Terra (sahibinden, satılan istasyondan) 80–150K TL.
- Spec: en az OCPP 1.6J Smart Charging Profile.

Bittiği nasıl anlaşılır: Charger labda, fiziksel teslim alındı.

Adım 62. EV emülatör / yük bankası

Niye: Gerçek araba kullanmak güvenlik+lojistik kabusu; emülatör DC çıkışı yükler.
Önce: Adım 62.
Ne yapacaksın:

- EV simülatör cihazı: comemso CCS 100, Keysight RP7900 (pahalı).
- Bütçe çözümü: DC yük direnci paneli + akım sınırlayıcı (5–15K TL).
- Riski: charger "araç bağlı değil" diye yazar; CP/PP dirençleriyle aldatma şart.

Bittiği nasıl anlaşılır: Charger "Charging" moduna geçiyor.

Adım 63. Charger'ı internet'e bağla

Niye: OCPP için CSMS URL'ine erişim lazım.
Önce: Adım 62.
Ne yapacaksın:

- Charger config menüsünden:
  - Network: DHCP veya statik IP.
  - OCPP: `ws://your-laptop-ip:9000/<charger_id>`.
  - Heartbeat interval: 60 sn.
- Lokal test için ngrok ya da port forwarding.

Bittiği nasıl anlaşılır: Charger ekranında "Connected" yazıyor.

Adım 64. İlk gerçek `BootNotification` — şampanya anı

Niye: Bu noktada teorik bilgi pratiğe dönüştü; semboliK olarak büyük adım.
Önce: Adım 64, Faz 3.
Ne yapacaksın:

- `evlb.ocpp.server` koş.
- Charger'ı aç → `BootNotification` log'a düşmeli.
- Vendor adı, modeli, firmware sürümü kaydet.

Bittiği nasıl anlaşılır: Log'da gerçek vendor (örn "Vestel") görünüyor; foto çek; ekibe paylaş.

Adım 65. İlk gerçek `StatusNotification`

Niye: Charger soketinin durumunu doğru okuyabiliyor muyuz.
Önce: Adım 65.
Ne yapacaksın:

- Soket boş → `Available`.
- Yük bankası takılı → `Preparing` veya `Charging`.
- Hata → `Faulted`.

Bittiği nasıl anlaşılır: 3 farklı durum log'a düştü.

Adım 66. İlk gerçek `StartTransaction`

Niye: Şarj başlatma akışını test.
Önce: Adım 66.
Ne yapacaksın:

- RFID kart oku (charger'da varsa) ya da app/QR.
- `Authorize` → `Accepted`.
- `StartTransaction` → `Accepted, transactionId=N`.

Bittiği nasıl anlaşılır: Transaction ID atandı.

Adım 67. Gerçek `MeterValues`

Niye: Anlık güç telemetrisi gerçek değer geliyor mu kontrol.
Önce: Adım 67.
Ne yapacaksın:

- Charger'a yük bağla.
- Her 60 sn `MeterValues` gelmeli.
- Power.Active.Import değerini multimetreyle çapraz doğrula.

Bittiği nasıl anlaşılır: Telemetry ±%5 doğru.

Adım 68. İlk gerçek `SetChargingProfile`

Niye: Power kontrolünün gerçekten çalıştığını kanıtla.
Önce: Adım 68.
Ne yapacaksın:

- Charger 11 kW çekiyorken `SetChargingProfile(limit=5500W)` gönder.
- 30 sn içinde MeterValues 5.5 kW'a düşmeli.
- Tekrar 22 kW limit gönder, yükselsin.

Bittiği nasıl anlaşılır: Charger limit'e uyuyor (kanıt: log + multimeter).

Adım 69. Power Quality Analyzer (Janitza UMG) bağla

Niye: Saha çıkışı gücü ölçecek (gerçek pilotta vazgeçilmez).
Önce: Adım 61.
Ne yapacaksın:

- Janitza UMG 605 (yaklaşık 35K TL) sipariş.
- Akım trafosu (CT) 100/5A 3 adet (her faza 1).
- Pano içinde montaj (elektrikçi).
- Modbus TCP ayarla.

Bittiği nasıl anlaşılır: Janitza ekranda 3 faz gerilim/akım gösteriyor.

Adım 70. Modbus TCP'den veri oku — Python

Niye: Janitza verisini yazılıma çekmeden işe yaramaz.
Önce: Adım 70.
Ne yapacaksın:

- `pip install pymodbus`
- `src/evlb/meter/janitza.py`:

```
from pymodbus.client import ModbusTcpClient

class JanitzaUMG605:
    def __init__(self, host, port=502):
        self.client = ModbusTcpClient(host, port)

    def read_total_power(self) -> float:
        # Janitza register 19026 = total active power
        result = self.client.read_holding_registers(19026, 2)
        return self._decode_float(result.registers)
```

- Janitza Modbus register tablosu (PDF dökümanlarında).

Bittiği nasıl anlaşılır: `print(meter.read_total_power())` → fiziksel yük ile uyumlu.

Adım 71. Harmonik (THD) okuma

Niye: Faz 4 diferansiyatörü için altyapı.
Önce: Adım 71.
Ne yapacaksın:

- Janitza THD_V, THD_I register'ları (her faz).
- `read_thd_voltage(phase: int)` ekle.

- Charger çalışıyorken/kapalıyken THD farkını ölç.

Bittiği nasıl anlaşılır: Charger DC modda → THD_I anlamlı yükseldi (diyelim %3 → %15).

Adım 72. IoT Gateway (Raspberry Pi)

Niye: Saha mahallinde verileri toplayıp buluta atan bir cihaz lazım.
Önce: Adım 71.
Ne yapacaksın:

- Raspberry Pi 4 (8GB) + 32GB SD + endüstriyel kasa (DIN ray).
- Raspberry Pi OS Lite (headless).
- SSH aç, Tailscale (mesh VPN) kur.
- Janitza IP'sine erişebilir mi test.

Bittiği nasıl anlaşılır: Pi'dan `ping janitza-ip` cevap veriyor.

Adım 73. MQTT broker — ingestion noktası

Niye: Telemetri akışı için standart kuyruk.
Önce: Adım 73.
Ne yapacaksın:

- Pi'ya `docker run -d -p 1883:1883 -p 9001:9001 eclipse-mosquitto`
- Lokalden `mosquitto_sub -h pi-ip -t '#'`.
- Pi'ya bir publisher script: Janitza'dan oku, MQTT'ye yayınla.

Bittiği nasıl anlaşılır: `mosquitto_sub` terminal'inde her saniye telemetry görüyorsun.

Adım 74. Endüstriyel switch + ağ izolasyonu

Niye: OT (charger, sayaç) ile IT (laptop, ofis) ağı ayrı olmalı.
Önce: Adım 61.
Ne yapacaksın:

- MOXA EDS-405A (5 portlu yönetilebilir).
- VLAN: VLAN 10 = OT (charger + Janitza), VLAN 20 = IT (Pi gateway).
- Pi sadece routing yapıyor.

Bittiği nasıl anlaşılır: Ofis laptop'un charger'a doğrudan ulaşamıyor, Pi üzerinden geçiyor.

7 FAZ 5 — Backend MVP (Adım 76–100)

Şimdi sadece çalışan bir bilgisayar değil, çoklu kullanıcılı bulut servisi inşa ediyoruz.

Adım 75. Mimari kararlar dokümanı

Niye: Stack seçimi sonradan ağrıyor; karar gerekçeli yazılı olmalı.
Önce: —
Ne yapacaksın:

- ADR (Architecture Decision Record) klasörü: `docs/adr/0001-backend-stack.md`.
- Karar: FastAPI + Postgres + Redis + Celery + TimescaleDB. Alternatifler ve ret nedenleri.

Bittiği nasıl anlaşılır: ADR dosyası repo'da.

Adım 76. FastAPI proje skeleton

Niye: REST API'nin temeli.
Önce: Adım 76.
Ne yapacaksın:

- Yeni repo: `evlb-api` (ya da monorepo'da `apps/api/`).
- `pip install fastapi uvicorn[standard] sqlmodel alembic httpx`
- `src/api/main.py`:

```
from fastapi import FastAPI

app = FastAPI(title="EV Load Balancing API")

@app.get("/health")
def health():
    return {"status": "ok"}
```

- `uvicorn api.main:app -reload`

Bittiği nasıl anlaşılır: `curl localhost:8000/health` → JSON dönüyor.

Adım 77. PostgreSQL kurulumu (Docker)

Niye: Lokal DB; üretimde aynısı.
Önce: —
Ne yapacaksın:

- `docker compose` dosyası oluştur (`infra/docker-compose.dev.yml`):

```
services:
  postgres:
    image: timescale/timescaledb:latest-pg16
    environment:
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: evlb
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

- `docker compose up -d`

Bittiği nasıl anlaşılır: `psql -h localhost -U postgres -d evlb` bağlanıyor.

Adım 78. SQLModel + Alembic migration

Niye: ORM + schema versiyonlama.
Önce: Adım 78.
Ne yapacaksın:

- `src/api/db/models.py`: `Tenant, Site, Charger, ChargingSession, MeterReading` tabloları.
- `alembic init` → ilk migration.
- `alembic revision -autogenerate -m "initial"`
- `alembic upgrade head`

Bittiği nasıl anlaşılır: Postgres'te tablolar görünüyor.

Adım 79. Multi-tenant kavramı (tenant_id sütunu)

Niye: Bir backend'de N müşteri.
Önce: Adım 79.
Ne yapacaksın:

- Her tablo: `tenant_id: UUID` sütunu, indexli.
- Postgres RLS (Row-Level Security) policy: "`tenant_id = current_setting('app.tenant_id')`".
- Test: tenant A'nın verisi tenant B'ye görünmüyor.

Bittiği nasıl anlaşılır: Manuel test geçiyor; yetkisiz select boş dönüyor.

Adım 80. İlk CRUD endpoint — Site

Niye: Müşteri "saha" kavramı; her şey buradan dallanıyor.
Önce: Adım 80.
Ne yapacaksın:

- `POST /sites`, `GET /sites`, `GET /sites/{id}`, `PATCH /sites/{id}`, `DELETE /sites/{id}`.
- Pydantic şemalar (`SiteCreate, SiteRead, SiteUpdate`).

Bittiği nasıl anlaşılır: Swagger `/docs` → 5 endpoint görünüyor; deneme POST'u sonra GET ile listede.

Adım 81. Charger CRUD

Niye: Site'in altındaki cihazlar.
Önce: Adım 81.
Ne yapacaksın:

- `POST /sites/{id}/chargers, ....`
- Charger fields: vendor, model, max_power_kw, ocpp_id (charger `BootNotification`'dan eşleşecek).

Bittiği nasıl anlaşılır: Swagger'dan charger ekleyip listeleyebiliyorsun.

Adım 82. Auth: Keycloak veya Auth0

Niye: Login olmadan API açık değildir; multi-tenant için kullanıcı/rol gerekli.
Önce: Adım 80.
Ne yapacaksın:

- Keycloak (self-hosted Docker) veya Auth0 (free 7K kullanıcı).
- OAuth2 flow: client credentials (M2M) + authorization code (kullanıcı).
- FastAPI middleware: `verify_jwt` dependency.
- Roller: `tenant_admin, operator, viewer`.

Bittiği nasıl anlaşılır: Auth header olmadan endpoint 401 dönüyor; valid token ile 200.

Adım 83. OCPP servisi backend ile entegrasyon

Niye: Faz 3'teki OCPP server'ı API ile birleştir.
Önce: Adım 60, 83.
Ne yapacaksın:

- Charger `BootNotification` → DB'de Charger kaydını bul (ocpp_id ile) → `accepted`/`rejected`.
- Yeni charger için "pending" durumu, admin onayı.
- `StatusNotification` → DB güncellenir.
- `MeterValues` → `MeterReading` tablosuna insert.

Bittiği nasıl anlaşılır: Sahte charger backend'e bağlanıyor, DB güncelleniyor.

Adım 84. TimescaleDB hypertable for telemetri

Niye: Dakikalık veri 1 sahada yılda 525K satır × 100 saha = 52M satır; düz Postgres yorulur.
Önce: Adım 84.
Ne yapacaksın:

- `SELECT create_hypertable('meter_readings', 'timestamp');`
- Kompresyon politikası: 30 günden eski veri sıkıştır.
- Retention: 24 ay sonra otomatik sil.

Bittiği nasıl anlaşılır: Hypertable görünüyor; chunked storage kullanıyor.

Adım 85. Redis kurulumu & cache pattern

Niye: Sık okunan veriler (config, son telemetri snapshot) DB'yi yormasın.
Önce: —
Ne yapacaksın:

- Docker compose'a Redis 7 ekle.
- `aioredis` client.
- Cache wrapper: `@cache(ttl=60)` decorator.

Bittiği nasıl anlaşılır: Endpoint ikinci çağrıda cache'den dönüyor (log).

Adım 86. Celery task queue

Niye: Algoritma her 60 sn çalışacak — async task gerekli.
Önce: Adım 86.
Ne yapacaksın:

- `pip install celery[redis]`
- `src/api/tasks.py`: `@celery.task` decorated fonksiyonlar.
- Worker: `celery -A api.tasks worker -beat`.
- İlk task: "her 60 sn her aktif site için `run_optimization`."

Bittiği nasıl anlaşılır: Worker log'unda her 60 sn task tetikleniyor.

Adım 87. Algoritma core'unu API'ye port et

Niye: Faz 1 sonu `evlb` paketi var; API onu kullanmalı.
Önce: Adım 30, 87.
Ne yapacaksın:

- API `requirements.txt`'a `evlb` ekle (path install).
- `run_optimization(site_id)` task: DB'den state oku → `DynamicFairController.allocate_power` çağır → OCPP service ile `SetChargingProfile` push.

Bittiği nasıl anlaşılır: Manuel `run_optimization(1)` → sahte charger'lara dağıtım gönderiliyor.

Adım 88. Logging — structured (JSON)

Niye: Production'da metin log değil, queryable JSON.
Önce: Adım 77.
Ne yapacaksın:

- `pip install structlog`
- Tüm log'lar JSON: `{ts, level, msg, tenant_id, site_id, ...}`.
- Request ID middleware (correlation ID).

Bittiği nasıl anlaşılır: Log'lar geçerli JSON.

Adım 89. Sentry entegrasyonu (error tracking)

Niye: Production hatasını görmek için.
Önce: Adım 89.
Ne yapacaksın:

- Sentry hesabı (free 5K events/ay).
- `sentry-sdk[fastapi]` kur.
- Test: kasıtlı exception → Sentry'de görünüyor.

Bittiği nasıl anlaşılır: Sentry dashboard'da test hatası.

Adım 90. API integration testi

Niye: CRUD + auth + OCPP zincirinin doğru aktığını otomatik kontrol.
Önce: Adım 82, 83, 88.
Ne yapacaksın:

- `pytest-asyncio + httpx`.
- Senaryo: tenant kayıt → site oluştur → charger ekle → sahte OCPP bağlanma → `run_optimization` → `SetChargingProfile` push.

Bittiği nasıl anlaşılır: `pytest tests/integration/` → tüm yeşil.

Adım 91. Tariff endpoint'leri

Niye: Faz 2'de yapılan modülü API'ye expose et.
Önce: Adım 39.
Ne yapacaksın:

- `POST /sites/{id}/calculate-roi` → JSON ROI raporu.
- `GET /tariffs` → kullanılabilir tarife listesi.

Bittiği nasıl anlaşılır: Swagger'dan ROI hesaplanıyor.

Adım 92. Telemetry query endpoint'leri

Niye: Dashboard verisi buradan beslenecek.
Önce: Adım 86.
Ne yapacaksın:

- `GET /sites/{id}/power?from=...&to=...&resolution=1m`.
- Pagination + max time range guards.

Bittiği nasıl anlaşılır: 24 saatlik veri ve 1 dk grain ile JSON dönüyor.

Adım 93. Alarm modeli (basit)

Niye: "Trafo aşımı", "Charger 30 dk heartbeat yok" gibi temel uyarılar.
Önce: Adım 88.
Ne yapacaksın:

- `Alarm` tablosu: type, severity, site_id, charger_id, message, created_at, resolved_at.
- Celery task: alarm condition kontrolü her 1 dk.

Bittiği nasıl anlaşılır: Sahte "charger offline" senaryosunda alarm DB'ye düşüyor.

Adım 94. E-posta bildirimleri

Niye: Müşteri push notification yok; en azından e-posta.
Önce: Adım 95.
Ne yapacaksın:

- SES (AWS) ya da SendGrid (free tier).
- Critical alarm → operator e-postası.

Bittiği nasıl anlaşılır: Test alarmı gerçek e-posta tetikliyor.

Adım 95. Webhook desteği

Niye: Müşterinin BMS / monitoring sistemine entegrasyon.
Önce: Adım 96.
Ne yapacaksın:

- Tenant başına `Webhook` kaydı (URL + secret).
- Alarm/transaction olaylarında HTTPS POST.
- HMAC-SHA256 signature.

Bittiği nasıl anlaşılır: Webhook.site'a deneme alarmı geliyor.

Adım 96. Audit log

Niye: "Kim, ne zaman, hangi setting'i değiştirdi."
Önce: Adım 80.
Ne yapacaksın:

- `AuditLog` tablosu: user_id, action, resource, old_value, new_value, ts.
- Middleware: tüm yazma işlemlerini loglar.

Bittiği nasıl anlaşılır: 1 site update sonra audit log'da kaydı.

Adım 97. API rate limiting

Niye: Kötü niyet ya da hatalı script DoS yapmasın.
Önce: Adım 87.
Ne yapacaksın:

- `slowapi` (Redis backend).
- Default: 60 req/min/user; M2M token için 600/min.

Bittiği nasıl anlaşılır: 70 hızlı request → 6 tanesi 429 dönüyor.

Adım 98. OpenAPI spec dışa aktarımı

Niye: Sözleşmeli müşteri "API dokümanı" ister; yatırımcı pitch'inde de.
Önce: Adım 81.
Ne yapacaksın:

- `python -c "from api.main import app; ..."` → `openapi.json`.
- Versiyonla: `docs/api/v1/openapi.json`.

Bittiği nasıl anlaşılır: `docs/api/` klasörü repo'da.

8 FAZ 6 — Telemetri Pipeline (Adım 101–115)

Adım 99. IoT Gateway'de production-grade publisher

Niye: Lab kurulumunu bir kez prodüktiv hale getir.
Önce: Adım 75.
Ne yapacaksın:

- Pi'da systemd service (`evlb-gateway.service`): boot'ta otomatik başla, crash'te restart.
- Konfigürasyon: `/etc/evlb/gateway.toml` (Janitza IP, MQTT broker, sample interval).

Bittiği nasıl anlaşılır: Pi reboot sonrası otomatik veri akıyor.

Adım 100. Bulut ingestion endpoint

Niye: Pi'dan gelen telemetriyi karşılayan API endpoint.
Önce: Adım 86, 101.
Ne yapacaksın:

- `POST /ingest/meter` (mTLS veya HMAC).
- Pi: 1 sn'lik buffer, batch insert.
- DB'ye TimescaleDB üzerinden `INSERT`.

Bittiği nasıl anlaşılır: `evlb-api`'de meter_readings tablosu büyüyor.

Adım 101. MQTT vs HTTP karar

Niye: Saha sayısı arttıkça HTTP ineye yorabilir.
Önce: Adım 102.
Ne yapacaksın:

- Karar: ilk 10 saha HTTP; sonra MQTT (HiveMQ Cloud, $50/ay).
- ADR yaz.

Bittiği nasıl anlaşılır: ADR repo'da.

Adım 102. Charger telemetry'sini birleşik pipeline'a al

Niye: OCPP MeterValues + Janitza farklı kaynaklar; tek tabloda birleşmeli.
Önce: Adım 90, 102.
Ne yapacaksın:

- `MeterReading.source` enum: `ocpp_charger, site_meter`.
- Aynı tablo, ayrı satırlar.

Bittiği nasıl anlaşılır: Query: `SELECT source, COUNT(*) FROM meter_readings GROUP BY source;`

Adım 103. Time-sync (NTP) doğru

Niye: Pi/charger/server saat farklı olunca telemetri patlar.
Önce: Adım 73.
Ne yapacaksın:

- Pi: `timedatectl set-ntp true`, NTP server: `tr.pool.ntp.org`.

- Charger: OCPP `Heartbeat` response'da `currentTime` field; charger saatini set eder.

Bittiği nasıl anlaşılır: Pi/charger/server arasında < 1 sn fark.

Adım 104. Veri kalite kontrol

Niye: Bozuk veri= bozuk model + bozuk ROI raporu.
Önce: Adım 102.
Ne yapacaksın:

- Ingestion sırasında kontrol: negatif güç, >1.5x trafo limiti, NaN, gelecek timestamp.
- Bozuk satır → `quarantine` tablosu, alarm.

Bittiği nasıl anlaşılır: Sahte bozuk veri ingestion → quarantine'e düşüyor.

Adım 105. Veri eksikliği detection

Niye: 5 dk telemetri yoksa charger offline sayılır.
Önce: Adım 95.
Ne yapacaksın:

- Celery task: her 1 dk her charger için son MeterValues kontrol.
- Eşik aşıldı → "offline" alarmı.

Bittiği nasıl anlaşılır: Sahte charger'ı kapat → 5 dk sonra alarm.

Adım 106. Background load = site_meter − charger_meter

Niye: Algoritma "baz yük" bilmek zorunda; tahmin modülü için input.
Önce: Adım 104.
Ne yapacaksın:

- `compute_background_load(site_id, ts)` = total_site_power − `SUM(charger_power)`.
- Aggregation per minute.

Bittiği nasıl anlaşılır: Time series view: site, charger toplam, baz yük.

Adım 107. Hava verisi entegrasyonu

Niye: Yük tahmininin önemli özniteliği.
Önce: —
Ne yapacaksın:

- OpenWeather (free 1K req/gün) veya MGM (resmi, free).
- Celery task: her 1 saat her saha için sıcaklık, ışık, nem topla.
- `WeatherReading` tablosu.

Bittiği nasıl anlaşılır: 24 saat verisi DB'de.

Adım 108. TR resmi tatil takvimi

Niye: Yük tahmini özniteliği; AVM/lojistik tatilde sıfıra yakın yükler.
Önce: —
Ne yapacaksın:

- Python: `holidays` kütüphanesi (TR destek var).

- `is_holiday(date)` helper.
- Ramazan/bayram için ek manuel girdi (öğle saati AVM hareketli).

Bittiği nasıl anlaşılır: 2026 TR tatil tablosunu oluşturuyorsun.

Adım 109. EPİAŞ Şeffaflık API entegrasyonu

Niye: Saatlik TR ulusal tüketim — pre-training prior.
Önce: —
Ne yapacaksın:

- EPİAŞ Şeffaflık Platformu hesap aç (free).
- OAuth2 token al.
- `GET /electricity-service/v1/consumption/data/realtime-consumption`.
- Tarihi veri (1 yıllık) çek; `epias_hourly` tablosuna kaydet.

Bittiği nasıl anlaşılır: Pandas DataFrame içinde 8760 satır (1 yıl saatlik).

Adım 110. Veri ambarı katmanı (analitik)

Niye: OLTP ile analitik aynı DB'de olmaz; sorgu darbeleri yer.
Önce: Adım 102.
Ne yapacaksın:

- Hetzner / küçük ölçekte: aynı Postgres'te ayrı schema (`analytics`) + materialized views.
- Faz 3'te ClickHouse veya Snowflake'e geçiş kararı.

Bittiği nasıl anlaşılır: `analytics.daily_site_power` view sorgulanıyor.

Adım 111. Veri export (CSV / Parquet)

Niye: Akademik partner ya da müşteri "ham veri" ister.
Önce: Adım 102.
Ne yapacaksın:

- `POST /sites/{id}/export?format=parquet&from=&to=`.
- Async job: S3-uyumlu (MinIO) bucket'a yaz, signed URL döndür.

Bittiği nasıl anlaşılır: 1 günlük export $<$ 1 dk.

Adım 112. Veri sahipliği + KVKK uyum sözleşmesi

Niye: Pilot sözleşmesinde net olmalı.
Önce: —
Ne yapacaksın:

- Avukatla sözleşme şablonu: "müşteri= veri sorumlusu, biz= veri işleyen, anonim aggregat verisi araştırma için kullanılabilir."

Bittiği nasıl anlaşılır: Şablon repo `docs/legal/`'a (gizli kısım).

Adım 113. Backup & restore prosedürü

Niye: Veri kaybı = pilot'un bitmesi + güven kaybı.
Önce: Adım 102.
Ne yapacaksın:

- Cron: gece 03:00 `pg_dump` → S3-uyumlu bucket.
- Retention: 30 günlük günlük + 12 aylık aylık snapshot.
- Restore tatbikatı: 1 ay sonra başka bir DB'ye `pg_restore`.

Bittiği nasıl anlaşılır: Restore tatbikatı başarılı.

9 FAZ 7 — Yük Tahmini (Adım 116–140)

Bu fazda gerçek ML iş başlıyor. Sıfırdan başlıyorsan: önce baseline, sonra XGBoost, sonra LSTM.

Adım 114. ML kütüphaneleri kurulumu

Niye: Stack'i bir kez kur, sonra üretkene geç.
Önce: Adım 7.
Ne yapacaksın:

- `pip install scikit-learn xgboost lightgbm prophet pandas matplotlib jupyter`
- Faz 7 sonu için: `pip install pytorch transformers neuralprophet`

Bittiği nasıl anlaşılır: `import xgboost; xgboost.__version__` hatasız.

Adım 115. Jupyter notebook environment

Niye: Veri keşfi için interaktif ortam şart.
Önce: Adım 116.
Ne yapacaksın:

- `notebooks/` klasörü.
- `nbstripout` (commit'e çıktı dahil etme).

Bittiği nasıl anlaşılır: İlk `notebooks/00_setup.ipynb` açılıyor.

Adım 116. Public dataset — ENTSO-E TR indirme

Niye: Pre-training için 5+ yıllık saatlik TR tüketim.
Önce: —
Ne yapacaksın:

- `entsoe-py` kütüphanesi.
- API key al (transparency.entsoe.eu).
- TR yük: 2019-01-01 → bugün, saatlik.
- Parquet olarak kaydet.

Bittiği nasıl anlaşılır: 50K+ satır parquet.

Adım 117. ACN-Data (Caltech) indirme

Niye: EV şarj davranışı için public proxy.
Önce: —
Ne yapacaksın:

- ev.caltech.edu/dataset kayıt + indirme.
- Sessions CSV: arrival, departure, energy, max_power.
- Notebook: keşif analizi (saatlik geliş yoğunluğu, SoC dağılımı).

Bittiği nasıl anlaşılır: Notebook'ta histogram + heatmap.

Adım 118. Pilot saha veri toplamayı başlat

Niye: Public veri TR ve site-specific için yetersiz.
Önce: Adım 65–75.
Ne yapacaksın:

- Pilot saha gateway'i kur (Faz 4 prosedür).
- En az 30 gün "shadow mode" (kontrol etme, sadece veri biriktir).

Bittiği nasıl anlaşılır: 30 gün × 1440 dk = 43.2K satır pilot veri.

Adım 119. Baseline tahmin: "geçen haftanın aynı saati"

Niye: Akıl baseline; bunu yenmek bile birinci adım.
Önce: Adım 120.
Ne yapacaksın:

- `evlb/forecast/baseline.py`: `predict(history, horizon)` → 7 gün önceki aynı slot.
- Walk-forward CV: son 7 gün test, öncesi train, kayan pencere.
- MAPE 24h hesapla.

Bittiği nasıl anlaşılır: Notebook'ta MAPE rakamı (örn %18–25).

Adım 120. Feature engineering — temel öznitelikler

Niye: Doğru özellikler = düşük MAPE; yanlış model = boşa eğitim.
Önce: Adım 121.
Ne yapacaksın:

- `features.py`:

```
def make_features(df):
    df['hour'] = df.ts.dt.hour
    df['dow'] = df.ts.dt.dayofweek
    df['month'] = df.ts.dt.month
    df['is_weekend'] = df.dow >= 5
    df['is_holiday'] = df.ts.dt.date.map(holiday_map)
    df['lag_1h'] = df.power.shift(60)
    df['lag_24h'] = df.power.shift(60*24)
    df['lag_168h'] = df.power.shift(60*24*7)
    df['rolling_24h_mean'] = df.power.rolling(60*24).mean()
    return df
```

Bittiği nasıl anlaşılır: 12+ feature kolonu var.

Adım 121. Hava + takvim feature'larını join

Niye: Sıcaklık AVM klima yükünü etkiler; tatil yükünü düşürür.
Önce: Adım 122.
Ne yapacaksın:

- Hava DataFrame ile saatlik join.
- TR holidays ile join.
- Eksik veri (interpolation).

Bittiği nasıl anlaşılır: Final DataFrame: power + 18 feature.

Adım 122. XGBoost model V1

Niye: Tabular zaman serisinde XGBoost şampiyon.
Önce: Adım 123.
Ne yapacaksın:

- `xgboost.XGBRegressor(n_estimators=500, max_depth=6, ...)`
- Train/val split: TimeSeriesSplit (sklearn).
- Metrik: MAE, MAPE, RMSE.

Bittiği nasıl anlaşılır: MAPE 24h, baseline'dan en az %30 iyi.

Adım 123. Hyperparameter tuning — Optuna

Niye: Default'lar nadiren optimal.
Önce: Adım 124.
Ne yapacaksın:

- `pip install optuna`
- 50 trial Bayesian search.
- Best params kaydet.

Bittiği nasıl anlaşılır: MAPE %5+ iyileşme.

Adım 124. Quantile forecast (P10/P50/P90)

Niye: Deterministic tahmin yerine belirsizlik bandı — karar daha iyi.
Önce: Adım 124.
Ne yapacaksın:

- LightGBM `objective="quantile", alpha=0.1, 0.5, 0.9`.
- 3 ayrı model.

Bittiği nasıl anlaşılır: Görselleştirme: P10/P50/P90 bantları.

Adım 125. Walk-forward CV otomatik

Niye: Modeller karşılaştırması bilimsel olmalı.
Önce: Adım 125.
Ne yapacaksın:

- `src/evlb/forecast/cv.py`: TimeSeriesSplit kullanarak `evaluate(model, df, n_splits=10)`.
- Çıktı: ortalama MAE, std, kötü-fold listesi.

Bittiği nasıl anlaşılır: Tablo: 3 model × MAE, MAPE.

Adım 126. MLflow tracking

Niye: Hangi model hangi datasetle eğitildi, takip et.
Önce: Adım 124.
Ne yapacaksın:

- `mlflow ui`, server kur.
- Her eğitimde: params, metrics, artifacts log.

Bittiği nasıl anlaşılır: MLflow UI'da deneme listesi.

Adım 127. Model registry + versiyonlama

Niye: Production'da hangi modelin canlı olduğunu bil.
Önce: Adım 128.
Ne yapacaksın:

- MLflow Model Registry: `Staging, Production` stage.
- Yeni model promote akışı.

Bittiği nasıl anlaşılır: Production'da v0.3 etiketli model.

Adım 128. Inference servisi (FastAPI içinde)

Niye: Algoritma her 1 dk tahmin isteyecek.
Önce: Adım 129.
Ne yapacaksın:

- `src/api/forecast/service.py`: `predict(site_id, horizon_min)` fonksiyonu.
- Model warm-up (boot'ta yükle).
- P99 latency $<$ 100 ms hedef.

Bittiği nasıl anlaşılır: Load test 100 req/s, p99 $<$ 100 ms.

Adım 129. LSTM prototip (V3 araştırma)

Niye: Yatırımcı pitch'inde "Transformer kullanıyoruz".
Önce: Adım 124.
Ne yapacaksın:

- PyTorch Lightning.
- Tek katman LSTM, hidden=64, 7 günlük input.
- GPU eğitim (Lambda Labs / Colab Pro).

Bittiği nasıl anlaşılır: XGBoost'tan biraz iyi MAPE; akademik makaleye temel.

Adım 130. Temporal Fusion Transformer (TFT)

Niye: State-of-the-art; yatırımcı sevimli.
Önce: Adım 132.
Ne yapacaksın:

- `pip install pytorch-forecasting`.
- TFT model + DataLoader.
- Eğitim: 10+ saat GPU.

Bittiği nasıl anlaşılır: TFT MAPE rakamı, attention plot.

Adım 131. Online learning (rolling retrain)

Niye: Saha sezon değişiklikleri model drift; haftalık retrain gerek.
Önce: Adım 130.
Ne yapacaksın:

- Celery beat schedule: her Pazar 03:00 retrain.
- Yeni model staging → A/B test (1 hafta) → production.

Bittiği nasıl anlaşılır: Otomatik retrain çalışıyor; MLflow'da yeni run.

Adım 132. Drift detection

Niye: Sahada ani değişim olursa model bozulur; alarm gerek.
Önce: Adım 134.
Ne yapacaksın:

- `evidently` kütüphanesi.
- Günlük distribution drift testi (KS-test).
- Eşik üstü → alarm.

Bittiği nasıl anlaşılır: Yapay drift senaryosu alarm tetikliyor.

Adım 133. EV gelişi tahmini ayrı modeli

Niye: Saha baz yükünden bağımsız; farklı pattern.
Önce: Adım 130.
Ne yapacaksın:

- Hedef: sonraki 1 sa içinde gelecek araç sayısı (Poisson).
- Feature: hour, dow, recent arrival counts, weather.
- Model: XGBoost classifier (0/1/2/3+).

Bittiği nasıl anlaşılır: 1-saat ileri MAE arrival_count < 1.

Adım 134. SoC dağılım modeli

Niye: Algoritma için gelecek araç "başlangıç SoC"i bilinmeli.
Önce: Adım 137.
Ne yapacaksın:

- ACN-Data + pilot veri ile gelen SoC histogramı.
- Saatlik Beta distribution fit.

Bittiği nasıl anlaşılır: Saatlik beta parametreleri kaydedildi.

Adım 135. Tahmin → karar entegrasyonu

Niye: Tahmin tek başına işe yaramaz; algoritmanın input'u olacak.
Önce: Adım 130, 137.
Ne yapacaksın:

- `run_optimization(site_id)` task içinde:
  - Önce: `forecast.predict(site_id, 60)` → 1 saatlik baz yük tahmini.
  - Sonra: `controller.allocate_power(...)` bu tahmini kullanarak.

Bittiği nasıl anlaşılır: Algoritma kararları tahmin-bilgili.

Adım 136. Tahmin doğruluğu reporting

Niye: Müşteri "model tutuyor mu?" diye sorar.
Önce: Adım 130.
Ne yapacaksın:

- Günlük: tahmin vs gerçek karşılaştırma → MAPE 1h, 6h, 24h.
- Aylık rapor PDF'ine ekle.

Bittiği nasıl anlaşılır: Aylık rapor sayfasında "forecast accuracy" grafiği.

10 FAZ 8 — Algoritmayı Production'a Port (Adım 141–150)

Adım 137. Algoritma servisi izole tut

Niye: "Optimization core" bir microservice; testi kolay, deploy ayrı.
Önce: Adım 88.
Ne yapacaksın:

- `src/optimizer/service.py`: `optimize(site_state, forecast, tariff)` → `ChargingPlan`.
- Pure function: state-in, plan-out.

Bittiği nasıl anlaşılır: Birim test edilebiliyor; backend bağımlılığı yok.

Adım 138. Rolling horizon framework

Niye: Şu anlık karar yerine 1-2 saatlik plan optimal.
Önce: Adım 141.
Ne yapacaksın:

- Her 1 dk: 60 dk ileriye optimize et, ilk dakikayı uygula, gerisini at.
- Test senaryosu: tahmin değişince plan güncelleniyor mu.

Bittiği nasıl anlaşılır: 1 saatlik simülasyonda planın 60 kez yenilendiği log.

Adım 139. CVXPY ile MILP/QP formülasyonu

Niye: Greedy yerine optimum karar.
Önce: Adım 142.
Ne yapacaksın:

- `pip install cvxpy`
- Decision: $P_{i,t}$ matris (araç × dakika).
- Constraint: charger limit, trafo limit, energy hedef.
- Objective: maliyet + bekleme cezası.

Bittiği nasıl anlaşılır: Küçük örnek (3 araç, 30 dk) CVXPY çözüyor.

Adım 140. Solver performansı

Niye: 1 dk içinde çözüm vermeli; yoksa real-time olmaz.
Önce: Adım 143.
Ne yapacaksın:

- Profiling: 10 araç, 60 dk.
- GLPK (free) vs Gurobi (lisans, 100x hızlı).
- Gerekirse heuristik fallback.

Bittiği nasıl anlaşılır: 10 araç problemi < 5 sn çözüyor.

Adım 141. Heuristik fallback

Niye: Solver patlarsa karar boşa düşmesin.
Önce: Adım 143.
Ne yapacaksın:

- DynamicFairController'ı fallback olarak kullan.
- Try MILP → 10 sn timeout → fallback.

Bittiği nasıl anlaşılır: Yapay timeout testi → fallback aktif.

Adım 142. A/B test framework

Niye: Yeni algoritma vs eski karşılaştırma için.
Önce: Adım 141.
Ne yapacaksın:

- Site config'inde `algorithm_variant: A | B`.
- Aynı saha günleri rastgele A/B'ye atanabilir.
- Sonuç: ortalama bekleme, aşım dakikası, fatura tasarrufu.

Bittiği nasıl anlaşılır: Test çıktı tablosu.

Adım 143. Dry-run mode

Niye: Pilotta önce "öneri yapıyor ama uygulamıyor".
Önce: Adım 141.
Ne yapacaksın:

- Site config: `control_mode: shadow | active`.
- Shadow modda `SetChargingProfile` gönderilmez, sadece log.

Bittiği nasıl anlaşılır: Pilot ilk 4 hafta shadow.

Adım 144. Algoritma KPI dashboard

Niye: İçerideki ekip için "algoritma sağlığı".
Önce: Adım 141.
Ne yapacaksın:

- Grafana panel: solver runtime, fallback rate, problem size.

Bittiği nasıl anlaşılır: Panel canlı veri gösteriyor.

Adım 145. Algoritmanın yan etkileri ölçümü

Niye: Yavaş şarj → batarya iyi, müşteri yavaşlık şikayeti edebilir.
Önce: Adım 146.
Ne yapacaksın:

- KPI: ortalama şarj süresi, tamamlanma oranı, hedef SoC altında bırakma yüzdesi.

Bittiği nasıl anlaşılır: Aylık raporda KPI'lar.

Adım 146. Algoritma decision log

Niye: Müşteri "bu araç niye yavaş şarjlanıyor?" diye sorduğunda cevap.
Önce: Adım 141.
Ne yapacaksın:

- Her decision'da: `decision_id, ts, inputs (state, forecast, tariff), output (plan), rationale`.
- DB'ye kayıt; 90 gün retention.

Bittiği nasıl anlaşılır: Bir transaction ID için decision history sorgulanabiliyor.

11 FAZ 9 — Operatör Dashboard (Adım 151–165)

Adım 147. Next.js proje skeleton

Niye: Frontend stack'i.
Önce: —
Ne yapacaksın:

- `npx create-next-app@latest evlb-dashboard -typescript -tailwind -app`
- shadcn/ui kurulumu.
- TanStack Query + Zustand (state).

Bittiği nasıl anlaşılır: `npm run dev` → `localhost:3000`.

Adım 148. Auth flow (Keycloak/Auth0 ile entegrasyon)

Niye: Login olmadan dashboard yok.
Önce: Adım 83.
Ne yapacaksın:

- NextAuth.js veya Auth0 SDK.
- Login sayfası → callback → token cookie.

Bittiği nasıl anlaşılır: Login flow uçtan uca çalışıyor.

Adım 149. API client

Niye: Backend ile konuşmak için tek noktadan.
Önce: Adım 100.
Ne yapacaksın:

- OpenAPI'den TypeScript client generate (`openapi-typescript`).
- React hook'lar: `useSites(), useChargers()`.

Bittiği nasıl anlaşılır: `useSites()` hook'u sites listesi dönüyor.

Adım 150. Sites listesi sayfası

Niye: İlk müşteri yüzü.
Önce: Adım 153.
Ne yapacaksın:

- Tablo: site adı, charger sayısı, son aktivite, alarm sayısı.
- Search + sort.

Bittiği nasıl anlaşılır: 5 site listede.

Adım 151. Site detay sayfası

Niye: Saha yöneticisinin ana ekranı.
Önce: Adım 154.
Ne yapacaksın:

- Tab'lar: Genel Bakış, Charger'lar, Tahmin, Raporlar, Ayarlar.

Bittiği nasıl anlaşılır: Tab navigasyonu çalışıyor.

Adım 152. Genel Bakış paneli

Niye: Anlık durum tek bakışta.
Önce: Adım 92, 155.
Ne yapacaksın:

- Üst: anlık güç (kW), trafo limit, kapasite kullanım yüzdesi.
- Orta: 24 saat power timeseries chart (Recharts).
- Sağ: aktif şarj sessions tablosu.

Bittiği nasıl anlaşılır: Sayfa yenilenmeden veriler güncelleniyor (1-dk WebSocket veya polling).

Adım 153. Real-time updates — Server-Sent Events (SSE)

Niye: Dashboard 1 dk "hard reload" kötü UX.
Önce: Adım 156.
Ne yapacaksın:

- Backend: `GET /sites/{id}/stream` (SSE endpoint).
- Frontend: `EventSource` hook.

Bittiği nasıl anlaşılır: Telemetri 1 sn'lik gecikmeyle güncelleniyor.

Adım 154. Charger'lar tab'ı

Niye: Cihaz bazlı durum ve kontrol.
Önce: Adım 155.
Ne yapacaksın:

- Grid view: her charger için card (durum, anlık güç, transaction).
- Detay modal: meter values geçmişi, son 24 saat session'lar.
- Buton: "Reset charger" (RemoteStop).

Bittiği nasıl anlaşılır: Charger card UI'ı dinamik.

Adım 155. Tahmin tab'ı

Niye: "Önümüzdeki 24 saat ne olacak?" — planlama için.
Önce: Adım 130, 155.
Ne yapacaksın:

- Chart: P10/P50/P90 bandı + tahmini araç gelişi.

Bittiği nasıl anlaşılır: Belirsizlik bandı görselleşmiş.

Adım 156. Raporlar tab'ı

Niye: Aylık ROI raporu burada indirilir.
Önce: Adım 40, 155.
Ne yapacaksın:

- Tarih aralığı seçici.
- "PDF rapor üret" butonu (ROI özet, tasarruf grafiği).

Bittiği nasıl anlaşılır: Test PDF download oluyor.

Adım 157. Alarm/Olay merkezi

Niye: Operatör "ne oldu?" diye hızlı görmeli.
Önce: Adım 95.
Ne yapacaksın:

- Sayfa: tüm alarm listesi (severity filtreli).
- Detay: timeline, ilgili charger, çözümleme.

Bittiği nasıl anlaşılır: Alarm listesi + filtre çalışıyor.

Adım 158. Ayarlar sayfası

Niye: Operatör algoritma davranışını ayarlamak ister (örn priorite kuralları).
Önce: Adım 88.
Ne yapacaksın:

- Algoritma seçici (Yönetimli / SRPT / Su Doldurma / Dinamik Adil).
- Eşik ayarlar (trafo emniyet payı, minimum keep-alive kW).

Bittiği nasıl anlaşılır: Setting save sonra optimizer yeni değerle çalışıyor.

Adım 159. Multi-tenant theming

Niye: Beyaz etiket potansiyeli; her CPO/EnerjiSA kendi logosuyla görmek ister.
Önce: Adım 80.
Ne yapacaksın:

- Tenant config: logo URL, primary color, name.
- Subdomain routing: `cpox.evlb.app`.

Bittiği nasıl anlaşılır: Test subdomain farklı tema yüklüyor.

Adım 160. Mobile responsive

Niye: Saha yöneticisi sahada telefonla bakar.
Önce: Adım 156.
Ne yapacaksın:

- Tailwind responsive classes ile ana sayfaları düzenle.
- Touch-friendly butonlar.

Bittiği nasıl anlaşılır: 360px viewport'ta okunabilir.

Adım 161. i18n (TR/EN)

Niye: TR ana, EN MENA / yatırımcı için.
Önce: Adım 154.
Ne yapacaksın:

- `next-intl`.
- Tüm string'leri `translations/tr.json, en.json`.

Bittiği nasıl anlaşılır: Dil değiştirici çalışıyor.

12 FAZ 10 — Bulut Deploy + CI/CD (Adım 166–175)

Adım 162. Hetzner Cloud hesabı + Terraform

Niye: Manuel server kurmak hata yapar.
Önce: Adım 76.
Ne yapacaksın:

- `hetzner-cloud-tr-resmi-iletim` hesap aç.
- `infra/terraform/` dizini.
- Resources: 1 server (CX22, 4GB RAM), 1 volume (40GB).

Bittiği nasıl anlaşılır: `terraform apply` → server canlı.

Adım 163. Docker image build

Niye: Reproducible deploy.
Önce: Adım 77.
Ne yapacaksın:

- `Dockerfile` (multi-stage: builder + runner).
- `.dockerignore`.
- GitHub Actions: build + push to ghcr.io.

Bittiği nasıl anlaşılır: Image registry'de.

Adım 164. Docker Compose production

Niye: Tek server'da hızlı başlangıç.
Önce: Adım 167.
Ne yapacaksın:

- `infra/docker-compose.prod.yml`: api + worker + postgres + redis + nginx (reverse proxy + Let's Encrypt).

Bittiği nasıl anlaşılır: `https://api.evlb.app` canlı.

Adım 165. CI/CD — GitHub Actions deploy

Niye: Manual deploy hata yapar.
Önce: Adım 167, 168.
Ne yapacaksın:

- `.github/workflows/deploy.yml`: `main` push → SSH server → `docker compose pull && up -d`.
- Slack/Discord notify.

Bittiği nasıl anlaşılır: Push → 5 dk içinde production yenileniyor.

Adım 166. Cloudflare DNS + SSL + WAF

Niye: DDoS koruması, ücretsiz SSL.
Önce: Adım 168.
Ne yapacaksın:

- Domain Cloudflare'e DNS bağla.

- Proxy ON.
- WAF rules.

Bittiği nasıl anlaşılır: `api.evlb.app` HTTPS.

Adım 167. Monitoring — Grafana + Prometheus + Loki

Niye: Production'da ne oluyor görmeden çalıştırma.
Önce: Adım 168.
Ne yapacaksın:

- Grafana Cloud (free tier) ya da self-host.
- Prometheus exporter: FastAPI metrics, Postgres metrics, Redis.
- Loki: log aggregation.

Bittiği nasıl anlaşılır: Grafana dashboard'unda CPU/RAM/req/s.

Adım 168. PagerDuty / Opsgenie alarm

Niye: Gece 3'te sistem patlarsa biri uyanmalı.
Önce: Adım 171.
Ne yapacaksın:

- Free Opsgenie 5 user.
- On-call rotation: 3 kurucu haftalık dönüşüm.
- Alarm: response time > 1 sn, error rate > %5, DB unreachable.

Bittiği nasıl anlaşılır: Test alarmı SMS/aramayla geliyor.

Adım 169. Backup automation

Niye: Disaster için (Faz 6'daki manual prosedürün otomasyonu).
Önce: Adım 115.
Ne yapacaksın:

- Cron container: gece 03:00 `pg_dump` → S3 (Hetzner Storage Box).
- Aylık restore tatbikatı (yeni VM, restore, smoke test).

Bittiği nasıl anlaşılır: Otomatik backup/restore çalışıyor.

Adım 170. Staging environment

Niye: Production'a önce staging'de dene.
Önce: Adım 169.
Ne yapacaksın:

- İkinci Hetzner server (CX22).
- `develop` branch → staging deploy.
- Staging'de manuel test geçmeden production'a gitme kuralı.

Bittiği nasıl anlaşılır: `staging.evlb.app` canlı.

Adım 171. Synthetic monitoring

Niye: "API çalışıyor mu?" soru bile vakit alıyor.
Önce: Adım 171.
Ne yapacaksın:

- Uptimerobot (free 50 monitor).
- Endpoint: `/health` her 5 dk.
- OCPP heartbeat sentetik bağlantı (her 10 dk).

Bittiği nasıl anlaşılır: Uptimerobot dashboard yeşil.

13 FAZ 11 — Pilot Saha Kurulumu (Adım 176–180)

Aylar süren teknik hazırlığın gerçek dünyaya çıktığı an. İlk sahayı çok dikkatli kur, çünkü ilk referans odur.

Adım 172. Saha öncesi teknik denetim

Niye: Sürpriz = kötü ilk izlenim.
Önce: Adım 175.
Ne yapacaksın:

- Saha ziyareti: trafo gücü, mevcut chargerlar (vendor, model, OCPP versiyonu), ağ altyapısı, sayaç tipi.
- Risk listesi (kablo eksiği, sayaç eski, charger firmware antika).
- Ön teknik rapor (1-2 sayfa).

Bittiği nasıl anlaşılır: Müşteri bu raporu onaylıyor.

Adım 173. Saha gateway donanım hazırlığı

Niye: Pi + Janitza + switch sahaya gitmeden lab'da test edilmeli.
Önce: Adım 75.
Ne yapacaksın:

- "Saha kit" — DIN ray pano, Pi gateway, Janitza, switch, kablolar.
- Lab'da bir gün full smoke test.
- Kargo + saha yer keşif.

Bittiği nasıl anlaşılır: Kit kargoda; kurulum tarihi sabit.

Adım 174. Saha kurulum (1 gün)

Niye: Elektrikçi + bizim ekibimiz; aksamasın.
Önce: Adım 177.
Ne yapacaksın:

- AG pano içine Janitza takılır (CT'ler ana hatta clamp).
- Pi gateway DIN ray.
- Switch ile charger ağı izole.
- Charger'ların OCPP URL'i bizim CSMS'e yönlendirilir.

Bittiği nasıl anlaşılır: Tüm chargerlar BootNotification gönderiyor; Janitza Modbus okunuyor; bulutta veri akıyor.

Adım 175. Shadow mode (ilk 4 hafta)

Niye: Kontrol etme, sadece veri biriktir; algoritma kararları "öneri" modunda.
Önce: Adım 178.
Ne yapacaksın:

- Site config: `control_mode=shadow`.
- Algoritma çıktıları DB'ye log'lanır, charger'a gönderilmez.

- Haftalık "öneriler dosyası" müşteriyle paylaş.

Bittiği nasıl anlaşılır: 4 hafta sonra:

- En az 30K dakikalık güvenilir veri var.
- Tahmin modeli site-specific fine-tune edildi.
- Müşteri "algoritma kararları mantıklı" onayı verdi.

Adım 176. Control mode aç + ilk öncesi-sonrası raporu

Niye: Müşteriye gerçek değer kanıtı.
Önce: Adım 179.
Ne yapacaksın:

- Site config: `control_mode=active`.
- İlk 2 hafta yarı-yarıya A/B (gün bazlı): yarısı algoritma, yarısı baseline.
- 30 gün sonra ilk müşteri raporu:
  - Trafo aşım dakikası: $X \to Y$ ($-\%Z$).
  - Aylık tahmin tasarruf: TL W.
  - Ortalama bekleme süresi: $A \to B$ dk.
  - Charger başına servis: $C \to D$ araç/saat.

Bittiği nasıl anlaşılır: Müşteri raporu kabul + ilk ücretli sözleşme imza.

14 Sonraki 5 Adım — Faz 11 sonrası (özet)

180. adımdan sonra gelen ana iş:

- Faz 12: 2–5. müşteriye scale (her birinde Faz 11'i tekrar).
- Faz 13: ISO 27001 sertifikasyon.
- Faz 14: OCPP 2.0.1 desteği.
- Faz 15: Demand response modülü (BEDAŞ pilot).
- Faz 16: Harmonik analiz, PV/ESS optimizasyonu (V2G hazırlığı).

15 Genel Notlar

15.1 Paralelize Edilebilen Adımlar (3 Kurucu)

CTO/Algoritma	Embedded/OCPP/Saha	CEO/Operasyon/Satış
Faz 1 (refactor)	Faz 3 (OCPP öğrenme)	Customer discovery (paralel)
Faz 2 (tarife)	Faz 4 (donanım sipariş)	TÜBİTAK BiGG başvuru
Faz 5 (backend)	Faz 6 (telemetri pipeline)	Hukuk + sözleşme şablonları
Faz 7 (ML)	Faz 11 (saha kurulum)	Pre-seed pitch + yatırımcı görüşme
Faz 8 (algoritma)	Charger üreticileri pazarlık	ROI raporu satış pitch

15.2 Hangi Adımı Ne Zaman Yapacaksın — Bağımlılık Grafiği

Adımlar lineer numaralandırılmış olsa da bağımlılık dikkate alınırsa:

- **Önce sıraya bağlı kalan zincir:** 1–15 → 16–30 → 76–100 → 141–150.
- **Paralel girişler:** 31–40 (tarife), 41–60 (OCPP), 116–140 (ML).
- **Donanım kritik path:** 61–75 (charger sipariş 6 hafta termin) → 176–180.

15.3 Bittiği Nasıl Anlaşılır (Genel)

Her adımda 3 kontrol:

1. **Kabul kriteri (acceptance test):** bu adımda ne çalışmalı.
2. **Git commit:** `git log` ile takip.
3. **Linear / Notion:** status= Done.

Her hafta sonu **adım sayısı progress raporu**: "180 adımdan X tamamlandı." Yatırımcı update'inde de aynı rakam.

15.4 Sıkıştığında Ne Yaparsın

- **1 saat saplandın:** arkadaşına anlat (rubber-duck), çoğu zaman çözüm gelir.
- **4 saat saplandın:** Stack Overflow, GitHub Issues, ChatGPT/Claude ile pair-programming.
- **1 gün saplandın:** adımı geçici olarak `TODO` işaretle, sonraki bağımsız adıma geç. Cumartesi tekrar bak.
- **1 hafta saplandın:** adım çok büyük; alt-adımlara böl. Ya da bağımlılık yanlış.

15.5 Final Hatırlatma

- **Asla 3 adımı birden yapma.** Atomik.
- **Her commit küçük.** "İdeal sayının altı PR'da iki sayfa diff."
- **Test yazmadan ileri gitme.** Faz 1'den sonra her adım test'le birlikte.
- **Documentation= kod kadar önemli.** README + ADR + kod içi docstring.
- **İlerleme= adım sayısı.** "Bugün ne yaptın?" sorusuna "adım 47 ve 48'i bitirdim" demek.

180 adımı sırayla bitirirsen, 12 ay sonunda elinde
***çalışan bir bulut servisi, 1+ ödeyen pilot müşteri, rakipten farklılaşmış bir ürün** olur.*

Sıfırdan başlayan biri için bu pek mümkün değil — 6 ayda 90 adım gerçekçi.
Ama her adım atılmaz değil; sadece sıralı atılırsa atılabilir.

Sürüm v1, 26 Nisan 2026— Adım Adım Teknik Roadmap.